DSIP Lab Internal Exam – MATLAB Codes and Short Answers

Prepared using the topics and coding style shown in the first uploaded PDF. Where a question requires a numerical convention or has ambiguous notation, the assumption is stated in the note.

1. Generate DT sequences

```
clc; clear; close all;
n = -5:10;
u = @(n) double(n >= 0);

% Assumption from the printed notation:
x1 = ((-1/9).^n).*u(n) - ((-1/3).^n).*u(n-1);
x2 = 1 + (2.^n).*u(n-1);
x3 = (0.2.^n).*u(n-1);

figure;
subplot(3,1,1); stem(n,x1,'filled'); grid on; title('x1(n)');
subplot(3,1,2); stem(n,x2,'filled'); grid on; title('x2(n)');
subplot(3,1,3); stem(n,x3,'filled'); grid on; title('x3(n)');
xlabel('n');
```

Note: Check the exponents in the printed question against your original paper before running; the scan/OCR notation is small.

2. Linear and 6-point circular convolution using DFTs

```
clc; clear; close all;
x1 = [1 2 1 2];       % arrow marks n=0 at first sample
x2 = [5 4 3 4 5];     % arrow marks n=0 at first sample

% Linear convolution
y_linear = conv(x1,x2);

% 6-point circular convolution using DFTs
N = 6;
X1 = fft(x1,N);
X2 = fft(x2,N);
y_circular = ifft(X1.*X2,N);
y_circular = real(y_circular);

disp('Linear convolution:'); disp(y_linear);
disp('6-point circular convolution:'); disp(y_circular);
```

Note: The arrowed first element is treated as n=0.

3. Recover x(n) from first six values of a 10-point DFT

```
clc; clear; close all;
% X(0)...X(5) are supplied. For a real x(n), use conjugate symmetry.
X = zeros(1,10);
X(1:6) = [10, -2+3j, 3+4j, 2-3j, 4+5j, 12];
X(7) = conj(X(5));   % X(6) = conj(X(4)) in 1-based indexing
X(8) = conj(X(4));   % X(7) = conj(X(3))
X(9) = conj(X(3));   % X(8) = conj(X(2))
X(10)= conj(X(2));   % X(9) = conj(X(1))
x = ifft(X,10);
disp('Reconstructed x(n):'); disp(real(x));
k = 0:9;
figure;
subplot(2,1,1); stem(k,abs(X),'filled'); grid on; title('|X(k)|');
subplot(2,1,2); stem(k,angle(X),'filled'); grid on; title('Angle of X(k)');
xlabel('k');
```

Note: The given X(5)=12 must be real for a real sequence, which it is. Conjugate symmetry is used for the remaining DFT values.

4. FIR lowpass using Hamming and Hann/Hanning windows (length 7)

```
clc; clear; close all;
Fs = 1000;
fc = 250;
M = 7;
Wn = fc/(Fs/2);       % normalized cutoff = 0.5

b_hamming = fir1(M-1,Wn,hamming(M));
b_hann = fir1(M-1,Wn,hann(M));
```

```
figure; freqz(b_hamming,1,512,Fs);
title('FIR LPF using Hamming window');
figure; freqz(b_hann,1,512,Fs);
title('FIR LPF using Hann window');

disp('Hamming coefficients:'); disp(b_hamming);
disp('Hann coefficients:'); disp(b_hann);
```

Note: Requires Signal Processing Toolbox. The question's f0 is interpreted as the Nyquist range Fs/2.

5. DFT circular frequency shift: Y(k)=X(k-1) mod 8

```
clc; clear; close all;
x = [5 -4 3 zeros(1,5)];   % 8-point DFT of zero-padded 3-point sequence
X = fft(x,8);
% Y(k)=X(k-1) with 0-based k: one-bin circular shift to the right
Y = circshift(X,[0 1]);
y = ifft(Y,8);
disp('y(n):'); disp(y);
figure; subplot(2,1,1); stem(0:7,abs(Y),'filled'); grid on; title('|Y(k)|');
subplot(2,1,2); stem(0:7,angle(Y),'filled'); grid on; title('Angle Y(k)');
```

Note: Uses 0-based DFT indices as in the question; first sample of x is n=0.

6. Recover x(n) from shifted 4-point DFT

```
clc; clear; close all;
% Given X(k-1) mod 4 = [-2-3j, 10, -2+3j, 3]
% Convert shifted spectrum to X(k): X(k)=G(k+1), where G(k)=X(k-1)
G = [-2-3j, 10, -2+3j, 3];
X = circshift(G,[0 -1]);
x = ifft(X,4);
disp('X(k):'); disp(X);
disp('x(n):'); disp(x);
figure; subplot(2,1,1); stem(0:3,abs(X),'filled'); grid on; title('|X(k)|');
subplot(2,1,2); stem(0:3,angle(X),'filled'); grid on; title('Angle X(k)');
```

Note: Circular shift interpretation follows X(k-1) with indices modulo 4.

7. Butterworth digital IIR using impulse invariance

```
clc; clear; close all;
% Specs: Ap=3 dB, As=35 dB, wp=0.35*pi, ws=0.45*pi, T=1 s
T = 1; Fs = 1/T;
Ap = 3; As = 35;
wp = 0.35*pi; ws = 0.45*pi;
% Pre-warp-like analog edges for impulse-invariance design (approximate)
Wp = wp/T; Ws = ws/T;
[N,Wc] = buttord(Wp,Ws,Ap,As,'s');
[bs,as] = butter(N,Wc,'s');
[bz,az] = impinvar(bs,as,Fs);
Hz = tf(bz,az,T);
figure; freqz(bz,az,1024); title('Impulse-invariance Butterworth response');
figure; zplane(bz,az); title('Digital pole-zero plot');
disp('b coefficients:'); disp(bz); disp('a coefficients:'); disp(az);
```

Note: Impulse invariance can cause aliasing; use the exact method required by your lab instructor/toolbox version.

8. 4-factor interpolation and 3-factor decimation

```
clc; clear; close all;
n = 0:199;
Fs = 1; % normalized sample-rate reference
x = sin(2*pi*0.05*n) + sin(2*pi*0.075*n);

% Interpolate by 4, then decimate by 3
L = 4; M = 3;
xi = interp(x,L);
yi = decimate(xi,M);

figure;
subplot(3,1,1); stem(n,x,'.'); grid on; title('Original sum of sinusoids');
subplot(3,1,2); stem(0:length(xi)-1,xi,'.'); grid on; title('4-factor interpolation');
subplot(3,1,3); stem(0:length(yi)-1,yi,'.'); grid on; title('3-factor decimation after interpolation');
xlabel('Sample index');
```

Note: interp and decimate use Signal Processing Toolbox; decimate applies anti-alias filtering.

9. DFT circular frequency shift: Y(k)=X(k+1) mod 8

```
clc; clear; close all;
```

```
x = [3 -3 3 -3 zeros(1,4)];
X = fft(x,8);
% Y(k)=X(k+1), modulo 8: one-bin circular shift to the left
Y = circshift(X,[0 -1]);
y = ifft(Y,8);
disp('y(n):'); disp(y);
figure; subplot(2,1,1); stem(0:7,abs(Y),'filled'); grid on; title('|Y(k)|');
subplot(2,1,2); stem(0:7,angle(Y),'filled'); grid on; title('Angle Y(k)');
```

Note: The 4-point sequence is zero-padded to length 8 before calculating its 8-point DFT.

10. Given X(k-2), find x(n-1)

```
clc; clear; close all;
% Given G(k)=X(k-2) mod 4
G = [3, -2-3j, 10, -2+3j];
% G(k)=X(k-2), hence X(k)=G(k+2)
X = circshift(G,[0 -2]);
x = ifft(X,4);
% x(n-1) is a one-sample delay in time: circular shift right for 4-point sequence
x_delay = circshift(x,[0 1]);
disp('X(k):'); disp(X);
disp('x(n-1), circular 4-point interpretation:'); disp(x_delay);
figure; subplot(2,1,1); stem(0:3,abs(X),'filled'); grid on; title('|X(k)|');
subplot(2,1,2); stem(0:3,angle(X),'filled'); grid on; title('Angle X(k)');
```

Note: A finite 4-point sequence is interpreted circularly for the plotted delay. Confirm instructor's intended indexing.

11. Butterworth LPF by bilinear transformation

```
clc; clear; close all;
Fs = 2000; T = 1/Fs;
fp = 200; fsb = 600;
Rp = 1.938; Rs = 13.98;

wp = 2*pi*fp/Fs;
ws = 2*pi*fsb/Fs;
% Prewarp digital frequencies for bilinear transformation
OmegaP = (2/T)*tan(wp/2);
OmegaS = (2/T)*tan(ws/2);

[N,Wc] = buttord(OmegaP,OmegaS,Rp,Rs,'s');
[bs,as] = butter(N,Wc,'s');
[b,a] = bilinear(bs,as,Fs);

figure; freqz(b,a,1024,Fs); title('Digital Butterworth LPF');
figure; zplane(b,a); title('Pole-zero plot');
disp('Order:'); disp(N);
disp('b ='); disp(b); disp('a ='); disp(a);
```

Note: Uses the bilinear-transform method and prewarps the edge frequencies.

12. Cascade systems and shifted/scaled impulse

```
clc; clear; close all;
h1 = [-0.04 0.04 0.3 -0.6 0.3 0.04 -0.04];
h2 = [0.09 -0.12 -0.5 -0.5 0.12 0.09];

% Cascade response
hc = conv(h1,h2);
% (i) h3(n)=h1*h2*3delta(n): multiply cascade response by 3
h3a = 3*hc;
% (ii) h3(n)=h1*h2*delta(n-1): one-sample delay
h3b = [0 hc];

figure; subplot(2,1,1); stem(0:length(h3a)-1,h3a,'filled'); grid on; title('3 h1(n)*h2(n)');
subplot(2,1,2); stem(0:length(h3b)-1,h3b,'filled'); grid on; title('h1(n)*h2(n)*delta(n-1)');
% Transfer functions represented by numerator coefficient vectors
disp('Numerator for case (i):'); disp(h3a);
disp('Numerator for case (ii):'); disp(h3b);
figure; freqz(h3a,1,1024); title('Magnitude/phase response, case (i)');
figure; freqz(h3b,1,1024); title('Magnitude/phase response, case (ii)');
```

Note: For FIR impulse responses, the transfer-function denominator is 1. A delay adds a leading zero.

13. Linear and 6-point circular convolution from shifted DFTs

```
clc; clear; close all;
% Given X1(-k) mod 4 and X2(k-1) mod 4
G1 = [7 -1 -1 -1];       % X1(-k)
G2 = [0 6 0 -2];          % X2(k-1)
```

```
X1 = G1([1 4 3 2]);       % reverse frequency indices modulo 4
X2 = circshift(G2,[0 -1]);% X2(k)=G2(k+1)
x1 = ifft(X1,4);
x2 = ifft(X2,4);

y_linear = conv(x1,x2);
N = 6;
y_circular = ifft(fft(x1,N).*fft(x2,N),N);
disp('x1(n):'); disp(x1);
disp('x2(n):'); disp(x2);
disp('Linear convolution:'); disp(y_linear);
disp('6-point circular convolution:'); disp(real(y_circular));
```

Note: DFT shift notation is interpreted modulo 4; verify indexing convention used in class.

14. Find X(k+2) and X(k) for x(n)={5,4,3,4}

```
clc; clear; close all;
x = [5 4 3 4];
X = fft(x,4);
X_k_plus_2 = circshift(X,[0 -2]);
disp('X(k):'); disp(X);
disp('X(k+2) modulo 4:'); disp(X_k_plus_2);
figure; subplot(2,1,1); stem(0:3,abs(X),'filled'); grid on; title('|X(k)|');
subplot(2,1,2); stem(0:3,angle(X),'filled'); grid on; title('Angle X(k)');
```

Note: The second requested spectrum is X(k) itself; X(k+2) is a circular shift by two bins.

15. Pole-zero plot from y(n)=sum k=0..3 (-2)^k x(n-k)

```
clc; clear; close all;
% y(n)=x(n)-2x(n-1)+4x(n-2)-8x(n-3)
b = [1 -2 4 -8];
a = 1;
figure; zplane(b,a); grid on; title('Pole-zero plot');
disp('Zeros:'); disp(roots(b));
disp('Poles: none (FIR system)');
```

Note: The equation is FIR with H(z)=1-2z^-1+4z^-2-8z^-3.

16. Output y(1) from step response s(n)=2δ(n+1)+δ(n)+δ(n-1)

```
clc; clear; close all;
% h(n)=s(n)-s(n-1); s has values s(-1)=2, s(0)=1, s(1)=1
% Therefore h(-1)=2, h(0)=-1, h(1)=0, h(2)=-1
% x(n)=0.5^n u(n); y(1)=sum_k h(k)x(1-k)
h_n = [-1 0 1 2];  % n = [0 1 -1 -2] would be confusing; use explicit sum below
h_index = [-1 0 1 2];
h_value = [-1 0 -1 2]; % h(-1)=2, h(0)=-1, h(1)=0, h(2)=-1
x = @(n) (0.5.^n).*(n>=0);
k = [-1 0 1 2];
h = [2 -1 0 -1]; % aligned with k = [-1 0 1 2]
y1 = sum(h .* arrayfun(@(kk) x(1-kk),k));
disp('y(1) ='); disp(y1);
```

Note: Uses h(n)=s(n)-s(n-1), with step response samples as listed in the question.

17. X(k)=k+1, 0<=k<=7; find x(n) and sum from k=0..3 x(2n)

```
clc; clear; close all;
k = 0:7;
X = k + 1;
x = ifft(X,8);
n = 0:7;
disp('x(n):'); disp(x);
figure; stem(n,real(x),'filled'); grid on; title('x(n) from 8-point IDFT'); xlabel('n');
% Interpreting the printed sum as sum_{n=0}^3 x(2n)
sum_even = sum(x(2*(0:3)+1));
disp('sum of x(2n), n=0..3:'); disp(sum_even);
```

Note: The final summation's variable formatting is ambiguous; code interprets it as sum n=0 to 3 of x(2n).

18. Salt-and-pepper noise; 3x3 average and median filters

```
clc; clear; close all;
I = imread('cameraman.tif');
if size(I,3)==3, I = rgb2gray(I); end
I = im2double(I);
J = I;
rng(1);
num = round(0.05*numel(I));
```

```
idx = randperm(numel(I),num);
half = floor(num/2);
J(idx(1:half)) = 0;          % salt-and-pepper: pepper
J(idx(half+1:end)) = 1;      % salt

avg = imfilter(J,ones(3,3)/9,'replicate');
med = medfilt2(J,[3 3]);

figure;
subplot(2,2,1); imshow(I); title('Original');
subplot(2,2,2); imshow(J); title('5% Salt-and-pepper noise');
subplot(2,2,3); imshow(avg); title('3x3 Average filter');
subplot(2,2,4); imshow(med); title('3x3 Median filter');
```

Note: Uses MATLAB sample image cameraman.tif; Image Processing Toolbox required.

19. Difference equation impulse response and stability

```
clc; clear; close all;
% y(n)-0.75y(n-1)+0.125y(n-2)=x(n)+x(n-1)
b = [1 1];
a = [1 -0.75 0.125];
N = 40;
x = [1 zeros(1,N-1)];
h = filter(b,a,x);
figure; stem(0:N-1,h,'filled'); grid on;
title('Impulse response'); xlabel('n'); ylabel('h(n)');
figure; zplane(b,a); title('Pole-zero plot');
p = roots(a);
disp('Poles:'); disp(p);
if all(abs(p)<1)
    disp('System is stable (all poles inside unit circle).');
else
    disp('System is unstable.');
end
```

Note: Causal rational system stability is checked by whether all poles lie inside the unit circle.

20. Output y(1) from step response s(n)=2δ(n)+δ(n-1)+δ(n-2)

```
clc; clear; close all;
% s(0)=2, s(1)=1, s(2)=1; h(n)=s(n)-s(n-1)
% Thus h(0)=2, h(1)=-1, h(2)=0, h(3)=-1
% x(n)=0.3^n u(n)
x = @(n) (0.3.^n).*(n>=0);
% y(1)=sum_k h(k)*x(1-k); h has nonzero samples at k=0,1,3
y1 = 2*x(1) + (-1)*x(0) + 0*x(-1) + (-1)*x(-2);
disp('y(1) ='); disp(y1);
```

Note: Uses h(n)=s(n)-s(n-1), assuming a causal step response sequence.

Batch-1 Q1. Define image histogram

```
% MATLAB: display histogram of an image
I = imread('pout.tif');
figure; imshow(I); title('Input image');
figure; imhist(I); title('Image histogram');
```

Note: Theory: an image histogram counts how many pixels occur at each intensity level.

Batch-1 Q2. Compute u(n)*delta(n)

```
clc; clear; close all;
n = -5:5;
u = double(n>=0);
delta = double(n==0);
y = conv(u,delta);
stem(-10:10,y,'filled'); grid on; title('u(n) * delta(n)');
```

Note: Convolution identity: u(n)*δ(n)=u(n).

Batch-1 Q3. Z-transform of x(n)={3,4,-5,9,7} and ROC

```
clc; clear; close all;
x = [3 4 -5 9 7];   % assume n=0 at first element, causal finite sequence
syms z
Xz = 3 + 4*z^(-1) - 5*z^(-2) + 9*z^(-3) + 7*z^(-4);
disp('X(z) ='); disp(Xz);
disp('ROC for causal finite sequence: |z| > 0 (all finite z except z=0)');
```

Note: ROC assumes the first element corresponds to n=0 and the sequence is right-sided.

Batch-1 Q4. Relation between DTFT and Z-transform

```
% Numerical illustration: evaluate DTFT on unit circle
x = [1 2 3 4];
N = 512;
w = linspace(-pi,pi,N);
X = zeros(1,N);
n = 0:length(x)-1;
for m = 1:N
    X(m) = sum(x .* exp(-1j*w(m)*n));
end
plot(w,abs(X)); grid on; xlabel('omega (rad/sample)');
title('DTFT magnitude');
```

Note: Theory: DTFT is the Z-transform evaluated on the unit circle z=e^(jω), when the ROC includes the unit circle.

Batch-1 Q5. Butterworth IIR filter order

```
clc; clear; close all;
Rp = 2; Rs = 10;
Wp = 20; Ws = 30;   % analog angular frequencies
[N,Wc] = buttord(Wp,Ws,Rp,Rs,'s');
disp('Minimum Butterworth order:'); disp(N);
disp('Cutoff frequency:'); disp(Wc);
```

Note: Order is determined from passband/stopband edges and attenuation specifications; buttord computes the minimum order.

Batch-2 Q1. Improve contrast using histogram

```
clc; clear; close all;
I = imread('pout.tif');
J = histeq(I);
figure;
subplot(1,2,1); imshow(I); title('Original');
subplot(1,2,2); imshow(J); title('Histogram equalized');
figure; subplot(1,2,1); imhist(I); title('Original histogram');
subplot(1,2,2); imhist(J); title('Equalized histogram');
```

Note: Histogram equalization spreads intensity levels to improve contrast.

Batch-2 Q2. Compute u(n)*delta(n-1)

```
clc; clear; close all;
n = -5:5;
u = double(n>=0);
d1 = double(n==1);
y = conv(u,d1);
% With correct convolution indexing, result is u(n-1)
stem(-10:10,y,'filled'); grid on; title('u(n) * delta(n-1)');
```

Note: Convolution identity: u(n)*δ(n−1)=u(n−1).

Batch-2 Q3. Z-transform of x(n)=(1/3)^n u(n)

```
clc; clear; close all;
syms z
Xz = 1/(1 - (1/3)*z^(-1));
disp('X(z) ='); disp(Xz);
disp('ROC: |z| > 1/3');
```

Note: Uses the right-sided geometric sequence.

Batch-2 Q4. Map poles from s-plane to z-plane

```
clc; clear; close all;
% Bilinear-transform mapping: z = (1+s*T/2)/(1-s*T/2)
T = 1;
s_poles = [-1+2j, -1-2j];
z_poles = (1 + s_poles*T/2)./(1 - s_poles*T/2);
disp('s-plane poles:'); disp(s_poles);
disp('Mapped z-plane poles:'); disp(z_poles);
figure; zplane([],z_poles); title('Mapped poles in z-plane');
```

Note: This code demonstrates bilinear-transform pole mapping; other methods use different mappings.

Batch-2 Q5. Frequency warping

```
% Frequency-warping formula for bilinear transform
clc; clear; close all;
T = 1/2000;
wd = linspace(0,pi,500);            % digital rad/sample
```

```
Omega = (2/T)*tan(wd/2);             % analog rad/s corresponding frequency
plot(wd,Omega); grid on;
xlabel('Digital frequency (rad/sample)');
ylabel('Analog frequency (rad/s)');
title('Frequency warping in bilinear transformation');
```

Note: Theory: the bilinear transform maps frequency nonlinearly; prewarping compensates at specified edge frequencies.